

Phonebook Program

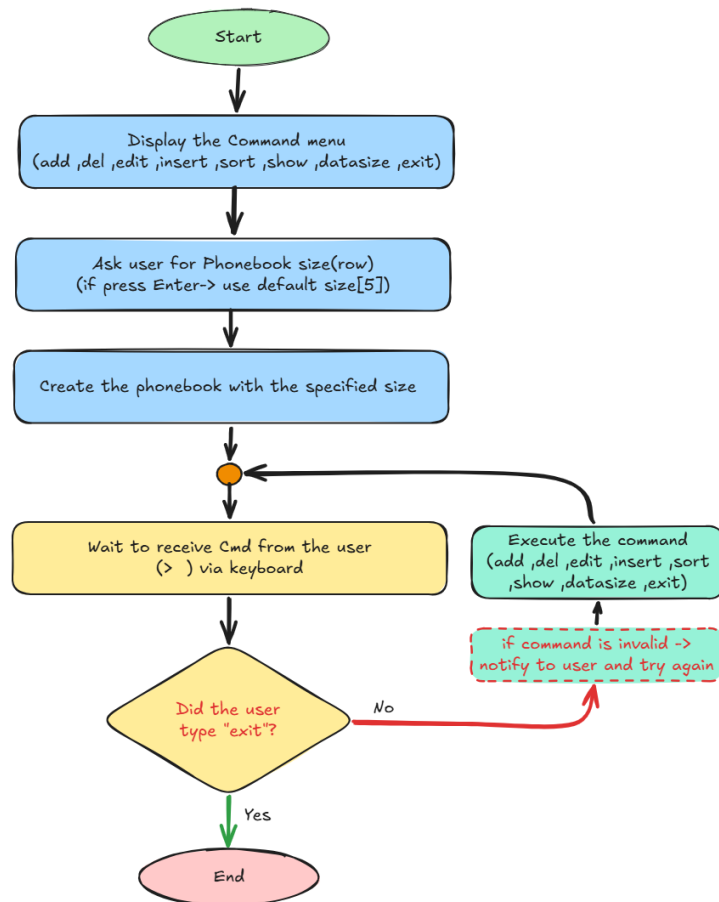
เอกสารออกแบบระบบ (System Design Document) — แนวคิด OOP

1. ภาพรวมของระบบ

Phonebook เป็นโปรแกรมจัดการสมุดโทรศัพท์อย่างง่าย ที่ออกแบบตามหลักการเขียน (Object-Oriented Programming: OOP) โดยแบ่งหน้าที่แต่ละ Class ออกจากกัน ประกอบด้วย 3 Class หลัก ได้แก่ Contact, Database และ List ที่มีหน้าที่เฉพาะของตัวเอง และเชื่อมโยงกันผ่านความสัมพันธ์แบบ Composition (has-a) โดยมีคลาส Main ในการรันโปรแกรม

2. ภาพรวมการทำงานของโปรแกรม (Flowchart คร่าวๆ)

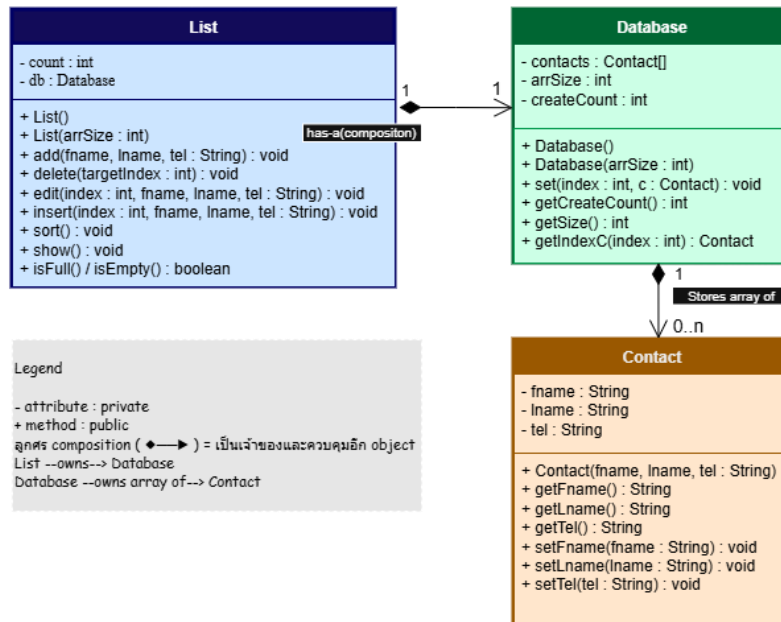
สรุปภาพรวมการทำงานของโปรแกรม (Class Main) แบบคร่าวๆ เพื่อให้เห็นลำดับการทำงานทั้งหมดก่อน โดยไม่ลงลึกถึง Code:



1. โปรแกรมจะ show cmd menu มีอะไรบ้างแล้วจะถามขนาดของ Phonebook (phonebook size) — หากไม่พิมพ์อะไรแล้วกด Enter จะใช้ค่าเริ่มต้น (default) เท่ากับ 5
2. หากพิมพ์ตัวเลข เช่น 10 โปรแกรมจะสร้าง Bookphone ที่เก็บได้สูงสุด 10 รายการ
3. จากนั้นจะรอรับค่าจากคีย์บอร์ด เพื่อรับ cmd แต่หากพิมพ์ข้อความที่ไม่ใช่ cmd โปรแกรมจะแจ้งเตือนและให้กรอกใหม่ จากนั้นก็ exe cmd และ ลูปถามคำสั่งจาก user ไปเรื่อยๆ ที่ละคำสั่งจนกว่า user จะพิมพ์ "exit" โปรแกรมถึงจะหยุดทำงาน

3. UML Class Diagram

แผนภาพด้านล่างแสดงโครงสร้าง Class ทั้ง 3 Class พร้อม attribute, method และความสัมพันธ์ระหว่าง Class (ไม่รวม Class Main เนื่องจากไม่ใช่ส่วนหนึ่งของโครงสร้างข้อมูลหลัก) :



4. รายละเอียดแต่ละคลาสและความเชื่อมโยง

4.1 Contact — หน่วยข้อมูล 1 รายการ

Contact คือคลาสที่เป็นตัวแทนของ "ข้อมูล 1 คน" ในสมุดโทรศัพท์ เปรียบเสมือน 1 row (record) ในตารางฐานข้อมูล ประกอบด้วยข้อมูล 3 อย่างคือ (Fname) (Lname) (Tel)

- **Role:** เก็บข้อมูลต่อ 1 คน
- **Encapsulation:** field ทั้งหมด (fname, lname, tel) เป็น private เข้าถึงได้ผ่าน getter/setter เท่านั้น
- **การเชื่อมโยงกับคลาสอื่น:** Contact ไม่ได้อ้างอิงไปหาคลาสอื่นแต่เป็น "ฝ่ายถูกเก็บ" โดย Class Database (Database ถือ array ของ Contact ไว้)

4.2 Database — (Data Storage)

Database คือคลาสที่ทำหน้าที่เป็น "คลังเก็บ" ของ Contact ทั้งหมด โดยใช้ array ชื่อ contacts (ชนิด Contact[]) เป็นโครงสร้างเก็บข้อมูล และมี arrSize กำหนดขนาดสูงสุดของ array

- **Role:** Access array ของ Contact ทั้งหมดแค่ class เดียว ไม่มี logic อยู่ในคลาสนี้เลย มีแค่ set() และ getIndexC()
- **Constructor Chaining:** Database() เรียก this(5) เพื่อส่งต่อไปยัง Database(int arrSize) ทำให้มีค่า default เท่ากับ 5
- **การเชื่อมโยงกับคลาสอื่น:** Database มีความสัมพันธ์แบบ Composition กับ Contact (Database "has-a" Contact[]) หรือก็คือ Database เป็นเจ้าของ array ของ Contact ทั้งหมด และเป็นคนที่ถูก List เรียกใช้งานอีกที (List "has-a" Database)

4.3 List — (Logic)

List คือ class ที่ทำหน้าที่เป็น control logic ทั้งหมดในการ Management เช่น add, delete, edit, insert, sort, และ show โดย List จะถือ Database เป็น field (private final Database db) แล้วเรียกใช้ db.set() / db.getIndexC() / db.getSize() เพื่อ access ข้อมูลแทน

- **Role:** Database ทุกคำสั่งที่กระทบข้อมูล (เพิ่ม/ลบ/แก้ไข/เรียงลำดับ) จะถูกควบคุมผ่าน class นี้ทั้งหมด
- **การเชื่อมโยงกับคลาสอื่น:** List "has-a" Database (composition) — เมื่อสร้าง List object จะสร้าง Database object เป็นของตัวเองไปพร้อมกัน (new Database(arrSize) ใน constructor) และ Database ก็ "has-a" Contact[] ต่ออีกทอดหนึ่ง ทำให้เกิดเป็นลำดับชั้น List → Database → Contact[] ซึ่งเป็นรูปแบบการออกแบบแบบ Composition

สรุปภาพรวม Relationship

- List เป็นเจ้าของ Database เพียงคนเดียว (1-to-1 composition) — เมื่อ List ถูกทำลาย Database ก็ตายไปด้วย
- Database เป็นเจ้าของ Contact หลายตัวผ่าน array (1-to-many composition) — Database ควบคุมการเข้าถึง Contact ทุกตัวแต่เพียงผู้เดียว ไม่มีใครแตะ array contacts ได้โดยตรงจากภายนอก == ex. Contact [c(fname,lname,tel) , c(fname,lname,tel) , c(fname,lname,tel)]
- Contact สามารถนำไปใช้ซ้ำในบริบทอื่นได้ในอนาคต
- Control flow : Main → List (Logic) → Database (Collection) → Contact (Raw data) และคืนค่ากลับในทิศทางตรงข้าม

5. Code trace /Call Flow form another class -- Example

5.1 class Contact

```

3 public class Contact {
4     private String Fname;
5     private String Lname;
6     private String Tel;
7
8     public Contact(String fname, String lname, String tel) {
9         this.Fname=fname;
10        this.Lname=lname;
11        this.Tel=tel;
12    }
13    //get
14    public String getFname() {return Fname;}
15    public String getLname() {return Lname;}
16    public String getTel() {return Tel;}
17    //set
18    public void setFname(String fname) {this.Fname=fname;}
19    public void setLname(String lname) {this.Lname=lname;}
20    public void setTel(String tel) {this.Tel=tel;}
21 }

```

ในคลาส Contact จะเห็นได้ว่าการ Encapsulation ที่ access ผ่านฟังก์ชัน getter & setter ที่ return ค่า เป็น String ข้อมูลออกไปตามฟังก์ชัน และในคลาสนี้สามารถเรียกใช้โดยการสร้าง object ที่มี argument คอยรับเมื่อมีการสร้างวัตถุนี้และ ค่า fname,lname,tel จะมีค่าตามที่รับผ่านทาง argument ที่เป็น constructor

ตัวอย่าง : การเรียกใช้ในคลาส Database ที่จะเก็บ obj ในคลาส Contact หลายๆ object

```
Contact[] contacts = new Contact[arrSize];
```

5.1 ภาพ class Contact

5.2 class Database

```

no commands
3  public class Database {
4      private Contact[] contacts ;
5      private int arrSize;
6
7      public Database () { //con
8          this.arrSize = 5;
9      }
10     public Database (int arrSize) { //Master con or Con Chaining
11         this.arrSize = arrSize;
12         this.contacts = new Contact[arrSize];
13     }
14     //set
15     public void set(int index, Contact c) {
16         this.contacts[index] = c;
17     }
18     //get
19     public int getSize() {return this.arrSize;}
20     public Contact getIndexC (int index) {return contacts[index];}
21 }

```

5.2 ภาพ class Database

ในคลาส Database เมื่อมีการสร้าง obj จะมีการรับ arrsize จากข้างนอก ที่เป็น argument ใน constructor และมอบเพื่อกำหนดขนาดของ array ที่เป็นคลังเก็บ obj ของ class Contact จากบรรทัด 11-12 หากไม่ได้กำหนดค่า arrsize จะให้ค่า default อยู่ที่ 5

-Function getSize(); ก็จะมีการ return ค่า arrsize ให้กับคลาสที่เรียกใช้

-Function getIndexC(); คือการ return ข้อมูลของ array contacts[]

เช่น มี array contacts[c("bell", "doe", "999-9"), c("JJR", "Tokien", "555-5")]
getIndexC(0); จะได้ชุดข้อมูล fname= "bell", lname= "doe", tel="999-9"

-Function set(); จะเป็นตัวการหลักในการเพิ่ม Contact เข้ามาในคลัง โดยมีการเก็บเป็น array ตามบรรทัดที่ 16 ถ้าจะเอาไปใช้จริงก็อยู่ใน logic การสร้าง, การสลับ หรือ การเลื่อนตำแหน่งของข้อมูล ใน Contact เช่น add, sort, delete, insert เป็นต้น

ตัวอย่าง : การเรียกใช้งาน function set(); ในคลาส List

5.2.1) การสร้าง

```

Database db = new Database(arrSize)
db.set(0, new Contact(fname, lname, tel));

```

5.2.2) การสลับ

```

Database db = new Database(arrSize)
Contact temp = db.getIndexC(j);
db.set(i, db.getIndexC(i + 1));
db.set(i + 1, temp);

```

5.2.3) การเลื่อนตำแหน่ง

```

Database db = new Database(arrSize)
for (int i=count-1; i>=index; i--) { //backward
    db.set(i + 1, db.getIndexC(i));
}

```

5.3 class List

```

3  public class List {
4      private int count;
5      private final Database db;
6      //con
7      public List () {
8          this(arrSize: 5);
9      }
10     public List (int arrSize) {
11         this.db = new Database(arrSize);
12     }

```

5.3 ภาพ class List

ในคลาส List จะเห็นได้ว่าการเรียกแค่ class Database ไม่มีการเรียก class Contact เนื่องจากให้ Database เป็นคนคอยจัดการให้

ใน constructor chaining จะมีการรับ argument จาก class Main มากำหนด arrsize เพื่อโอนค่าให้ Database แล้ว Database จะกำหนดขนาด array contacts[] ให้ตามภาพ 5.2 บรรทัดที่ 12 ซึ่งหมายความว่าตัว การสร้าง object Database เพียงตัวเดียวสามารถสร้าง object Contact หลายๆตัวถึงขนาด array ที่กำหนดได้

ตัวอย่าง : การเรียกใช้ในคลาส List

```

List list = new List(sizeData); //แบบกำหนดค่า
List list = new List(sizeData); //แบบ Default

```

- **add** — เพิ่มรายชื่อใหม่ ระบบจะถาม (First name) (Last name) และ (Tel.) ที่รับค่าเป็น String และเพิ่มจำนวนข้อมูล
- **delete** — ลบรายชื่อ โดยระบุลำดับที่ (No.) ของรายชื่อที่ต้องการลบตามที่แสดงในคำสั่ง show (ไม่ใช่ index ที่เป็น array) โดยการ del จะใช้วิธีการเลื่อนตำแหน่ง array ตั้งแต่ target ถึงจำนวนข้อมูลที่มีอยู่ ไปทางซ้ายอิงตาม vector นั้นหมายถึง ตัว target จะถูกเขียนทับ และทำการลดจำนวนข้อมูลลง
- **edit** — แก้ไขรายชื่อที่มีอยู่แล้ว โดยระบุลำดับที่ (No.) แล้วกรอกชื่อ นามสกุล และเบอร์โทรใหม่ที่ทับของเดิม เป็นการ overwrite
- **insert** — แทรกรายชื่อใหม่ โดยระบุลำดับที่ (No.) โดยรายชื่อเดิมตั้งแต่ตำแหน่งนั้นจะถูกเลื่อนถัดไปทั้งหมด ใช้วิธีการเลื่อนตำแหน่งเหมือน delete แต่ไปทางเลื่อนไปทางขวา และ เพิ่มชุดข้อมูลใหม่ลงไปตาม target และเพิ่มจำนวนข้อมูล
- **sort** — เรียงลำดับรายชื่อทั้งหมดตาม (First name) จาก A ไป Z โดยใช้วิธี Bubble Sort ในการ swap ชุดข้อมูล
- **show** — แสดงรายชื่อทั้งหมดที่มีอยู่ใน phonebook ขณะนั้น พร้อมลำดับที่ (No.)
- **datsize** — แสดงจำนวนครั้งที่เพิ่มเขียนข้อมูลลง Database นับสะสมตั้งแต่เริ่มโปรแกรม
- **exit** — ออกจากโปรแกรม

6. Source Code

6.1 Contact.java

```
public class Contact {  
    private String FName;  
    private String Lname;  
    private String Tel;  
  
    public Contact(String fname, String lname, String tel) { //input --->new  
        this.FName=fname;  
        this.Lname=lname;  
        this.Tel=tel;  
    }  
    //get  
    public String getFName() {return FName;}  
    public String getLname() {return Lname;}  
    public String getTel() {return Tel;}  
    //set  
    public void setFName(String fname) {this.FName=fname;}  
    public void setLname(String lname) {this.Lname=lname;}  
    public void setTel(String tel) {this.Tel=tel;}  
}
```

6.2 Database.java

```
public class Database {
    private Contact[] contacts ;
    private int arrSize;

    public Database () { //con
        this(5);
    }
    public Database (int arrSize) { //Master con or Con Chaining
        this.arrSize = arrSize;
        this.contacts = new Contact[arrSize];
    }
    //set
    public void set(int index, Contact c) {
        this.contacts[index] = c;
    }
    //get
    public int getSize() {return this.arrSize;}
    public Contact getIndexC (int index) {return contacts[index];}
}
```

6.3 List.java

```
public class List {
    private int count;
    private final Database db;
    //con
    public List () {
        this(5);
    }
    public List (int arrSize) {
        this.db = new Database(arrSize);
    }
    //-----funtion
    public void add (String fname, String lname, String tel) {
        if (!isFull()) {
            db.set(count, new Contact(fname, lname, tel));
            count++;
        } else System.out.println("The Array is Full.");
    }
    public void delete (int targetIndex) {
        if (!isEmpty()) {
            if (targetIndex >= 0 && targetIndex < this.count) {
                for (int i=targetIndex; i<this.count-1 ;i++) { //index-1 --> outofbound &&
for//move position
                    db.set(i, db.getIndexC(i+1)); //move
                }
                this.count--;
            } else System.out.println("Invalid index. Cannot delete.");
        }
    }
    public void edit (int indexs ,String fname, String lname, String tel) {
        if (!isEmpty()) {
            Contact c = db.getIndexC(indexs);
```

```

        c.setFname(fname);
        c.setLname(lname);
        c.setTel(tel);
    } else System.out.println("These index out of range the array.");
}
public void insert (int indexs,String fname, String lname, String tel) {
    if (!isFull()) {
        for (int i=count-1; i>=indexs;i--) { //move position backward
            db.set(i + 1, db.getIndexC(i));
        }
        db.set(indexs, new Contact(fname, lname, tel));
        count++;
    } else {
        System.out.println("Invalid index. Cannot insert.");
    }
}
public void sort () { //Bubble sort
    Contact temp;
    int size = this.count;
    for (int i=0; i<size-1 ; i++) {
        for(int j=0; j<size-1-i ; j++) {
            if (db.getIndexC(j).getFname().compareTo(db.getIndexC(j+1).getFname()) > 0) {
                temp = db.getIndexC(j);
                db.set(j, db.getIndexC(j + 1));
                db.set(j + 1, temp);
            }
        }
    }
}
//-----check
public boolean isFull() { return count == db.getSize(); }
public boolean isEmpty () { return this.count == 0;}
public void showCreateCount() {
    System.out.println("You have created " + this.count+ " of " + db.getSize());
}
public void show () {
    System.out.println("=====
=====");
    System.out.printf("%-5s %-15s %-15s %-15s\n", "No.", "First Name", "Last Name", "Phone
Number"); {
        if (!isEmpty()) {
            for (int i=0; i<count ;i++) {
                System.out.printf("%-5d %-15s %-15s %-15s\n", i+1,
db.getIndexC(i).getFname(), db.getIndexC(i).getLname(), db.getIndexC(i).getTel());
            }
        } else System.out.println("Array is empty.");
    }
}
}
}

```

6.4 Main

```

import java.util.Scanner;

public class Main {
    static int sizeData;
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            List list;
            String fname, lname, tel;
            int index;

            showInit();
            while(true) {
                System.out.print("Enter phonebook size or press Enter for default(5): ");
                String sizeStr = scanner.nextLine().trim();
                if (sizeStr.isEmpty()) {list = new List();sizeData = 5;break;}
                try {sizeData = Integer.parseInt(sizeStr);list = new List(sizeData);break;}
                catch (NumberFormatException e) {System.out.println("Invalid size.Please enter a
number or press Enter for default(5).");}
            }
            /*setup in database*/list.setUp();
            while(true) {
                System.out.print("> ");
                String input = scanner.nextLine().toLowerCase().trim();
                if (input.equals("exit")) {
                    System.out.println("Exit program.");
                    break;
                }
                switch (input) {
                    case "add" -> {
                        System.out.print("First name : ");fname = scanner.nextLine().trim();
                        System.out.print("Last name : ");lname = scanner.nextLine().trim();
                        System.out.print("Tel. : ");tel = scanner.nextLine().trim();
                        list.add(fname, lname, tel);
                    }
                    case "delete" -> {
                        index = indexInput(scanner);
                        list.delete(index-1);
                    }
                    case "edit" -> {
                        index = indexInput(scanner);
                        System.out.print("First name : "); fname = scanner.nextLine().trim();
                        System.out.print("Last name : "); lname = scanner.nextLine().trim();
                        System.out.print("Tel. : "); tel = scanner.nextLine().trim();
                        list.edit(index-1, fname, lname, tel);
                    }
                    case "insert" -> {
                        index = indexInput(scanner);
                        System.out.print("First name : "); fname = scanner.nextLine().trim();
                        System.out.print("Last name : "); lname = scanner.nextLine().trim();
                        System.out.print("Tel. : "); tel = scanner.nextLine().trim();
                        list.insert(index-1, fname, lname, tel);
                    }
                    case "sort" -> list.sort();
                    case "show" -> list.show();
                    case "datasize" -> list.showCreateCount();
                    default -> System.out.println("Please try again.");
                }
            }
        } catch (Exception e) {System.out.println("something went wrong.");}
    }
    static void showInit() {
        System.out.printf("%-6s %-10s %-10s\n", "Cmd :", "1.add", "2.delete");
    }
}

```



```
System.out.printf("%-6s %-10s %-10s\n", "", "3.edit", "4.insert");
System.out.printf("%-6s %-10s %-10s\n", "", "5.sort", "6.show");
System.out.printf("%-6s %-10s %-10s\n", "", "7.datasize", "8.exit");
}
static int indexInput(Scanner scanner) {
    int target = -1;
    boolean check = true;
    while (check) {
        System.out.print("Target No. : ");
        try {
            int inputInt = Integer.parseInt(scanner.nextLine().trim());
            if (inputInt >= 1 && inputInt <= sizeData) {
                target = inputInt;
                check = false;
            } else {System.out.println("Range is out of data size.");}
        } catch (NumberFormatException e) {System.out.println("Input numeric only."); }
    }
    return target;
}
}
```